

Algorithms and Data Structures

Sujeong Kim



Dynamic Programming (2)

Matrix Multiplication

Longest Common Subsequence

Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Optimization Problems

- Problems where a set of choices must be made to achieve an optimal (min/max) value, subject to some constraints.
- There may be multiple solutions that yield the same optimal value.
- Two common techniques:
 - Dynamic Programming (global approach)
 - Systematically explore all possible solutions and selects the best overall.
 - Greedy Algorithms (local)
 - Make the best local choice at each step, following a predefined rule.

Optimized Chain Operations

- Identify the most efficient order for executing a series of operations.
- This problem is important in both compiler design for code optimization and databases query optimization
 - Aim to minimize computational resources, such as time and memory.
 - Compiler Design: Optimizing the sequence of operations in code. Compilers aim to generate code that runs efficiently on the hardware.
 - Databases: Ensure that database queries execute as quickly as possible by finding the optimal sequence of operations for data retrieval.

Optimized Chain Operations – Matrix Chain Multiplication

- Properties of matrix multiplication
 - Associativity: $(AB)C = A(BC)$
 - Non-commutativity: $AB \neq BA$
 - Dimension compatibility: if A is $p \times q$, and B is $q \times r$, then:
 - The product AB is defined
 - The cost is $p \cdot q \cdot r$
 - The resulting matrix is $p \times r$

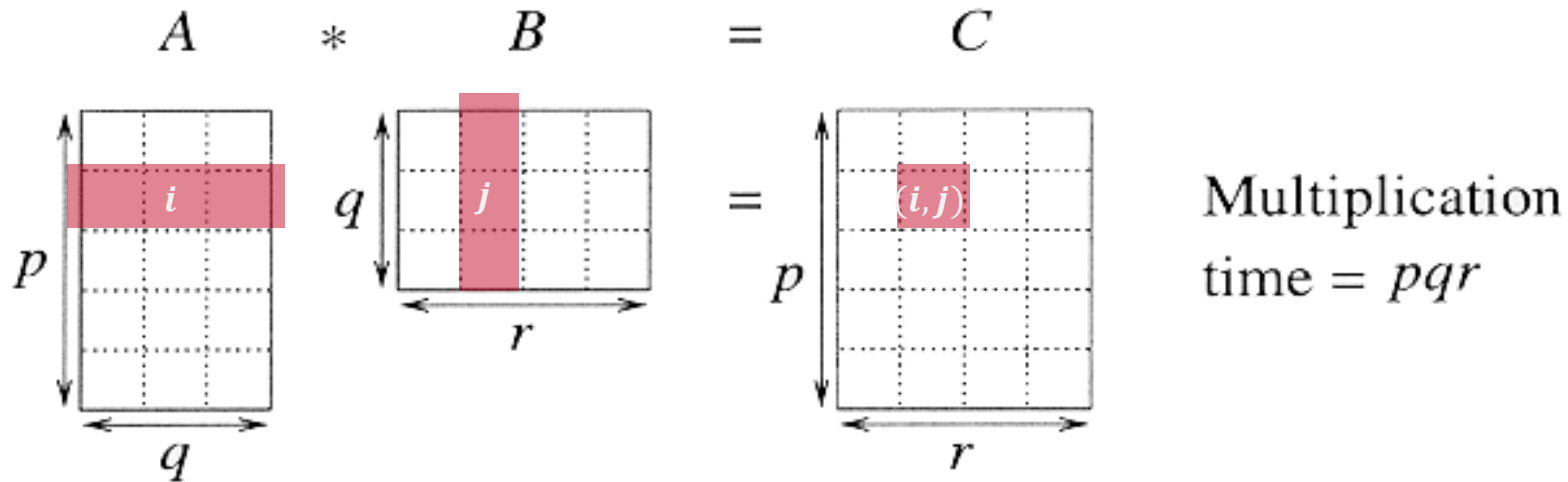
Matrix Chain Multiplication (MCM)

- Given a series of matrices: $A_1 A_2 \dots A_n$, we can place parentheses in many ways because matrix multiplication is associative (but not commutative).
- All parenthesizations produce the same final matrix, since the mathematical result doesn't change
- Different parenthesization can have dramatically different computational costs, depending on the order in which multiplications are performed
- MCM is the problem of finding the parenthesization that yields the minimum total cost

Matrix Multiplication

- Given two matrices:
 - A of dimensions $p \times q$
 - B of dimensions $q \times r$
- For matrix C , each element $C[i, j]$ (where $1 \leq i \leq p$ and $1 \leq j \leq r$) is computed as:

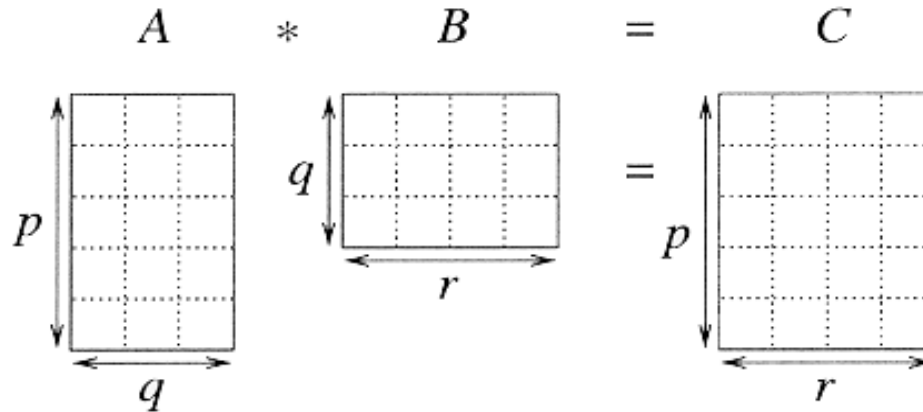
$$C[i, j] = \sum_{k=1}^q A[i, k] \cdot B[k, j]$$



Matrix Multiplication

- Observe that there are pr total entries in C
- Time to compute one element $C[i, j]$
 - Each element $C[i, j]$ requires q multiplications and q additions: $O(q)$
- Total Time Complexity
 - Since there are pr entries in C , the total time to compute the matrix multiplication is pqr

$$C[i, j] = \sum_{k=1}^q A[i, k] \cdot B[k, j]$$



Multiplication
time = pqr

Chain Matrix Multiplication

- Given a sequence of matrices $A_1 A_2 \dots A_n$, and dimensions $p_0 p_1 \dots p_n$ where A_i is of dimension $p_{i-1} \times p_i$, determine multiplication sequence that minimizes the number of operations (i.e., total number of scalar multiplication).
- This algorithm does not perform the multiplication, it just figures out the best order in which to perform the multiplication.
- What if each matrix is a square matrix of dimension $m \times m$? Multiplying two matrices is $O(m^3)$
- Fixed-order multiplication:
 - The cost of computing $(A_1 A_2 \dots A_n)$ is $O((n - 1)m^3) \approx O(nm^3)$

Example: Chain Matrix Multiplication

- Consider 3 matrices: A_1 be 5 x 4, A_2 be 4 x 6, and A_3 be 6 x 2.

$$\text{Mult}[(A_1 A_2) A_3] = (5 \times 4 \times 6) + (5 \times 6 \times 2) = 180$$

$$\text{Mult}[A_1 (A_2 A_3)] = (4 \times 6 \times 2) + (5 \times 4 \times 2) = 88$$

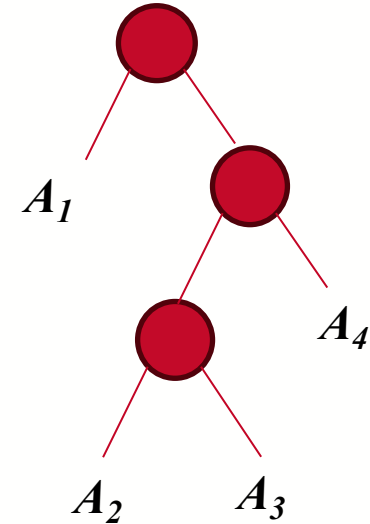
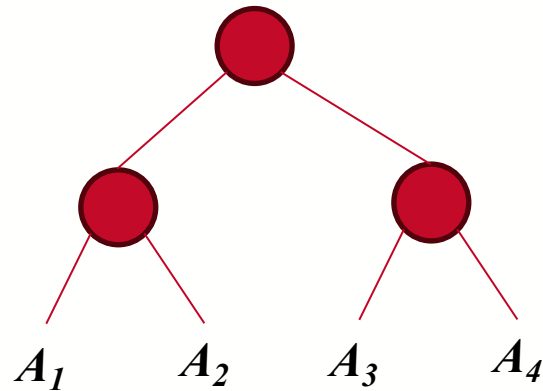
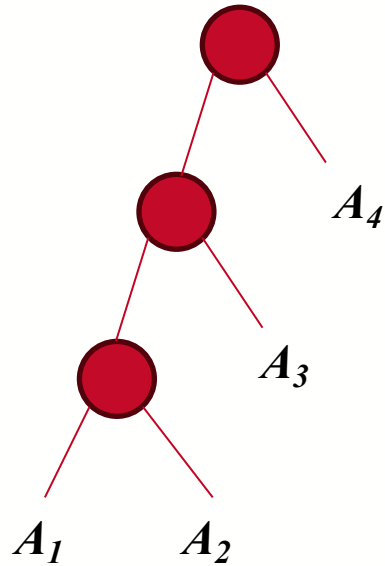
Even for this small example, considerable savings can be achieved by reordering the evaluation sequence.

Exercise:

- Consider multiplication of four matrices $A_1A_2A_3A_4$ with dimensions given by $\langle 1,2,3,4,3 \rangle$
 1. List all possible ways to fully parenthesize the product $A_1A_2A_3A_4$
 2. For each parenthesization, compute the total number of scalar multiplications

Exercise:

- $((A_1 A_2)A_3)A_4$
- $(A_1 A_2)(A_3 A_4)$
- $A_1((A_2 A_3)A_4)$
- ...



...

Cost of Naive Algorithm

- Try all valid parenthesizations to find the one with minimal scalar cost.
- This is related to *Catalan numbers*

$$C(n) = \frac{1}{n+1} \binom{2n}{n} \in \Omega(4^n / n^{3/2})$$

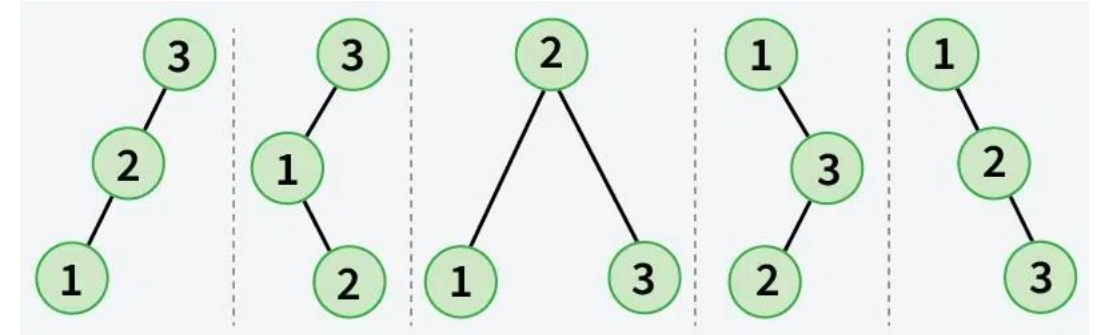
where $\binom{2n}{n}$ is a binomial coefficient, the number of ways to choose n items out of $2n$ items total.

- These numbers grow rapidly as n increases and are used in various combinatorial problems.
- The number of ways to parenthesize n matrices corresponds to the $(n - 1)$ th Catalan number: $P(n) = C(n - 1)$.

Catalan Numbers (Examples)

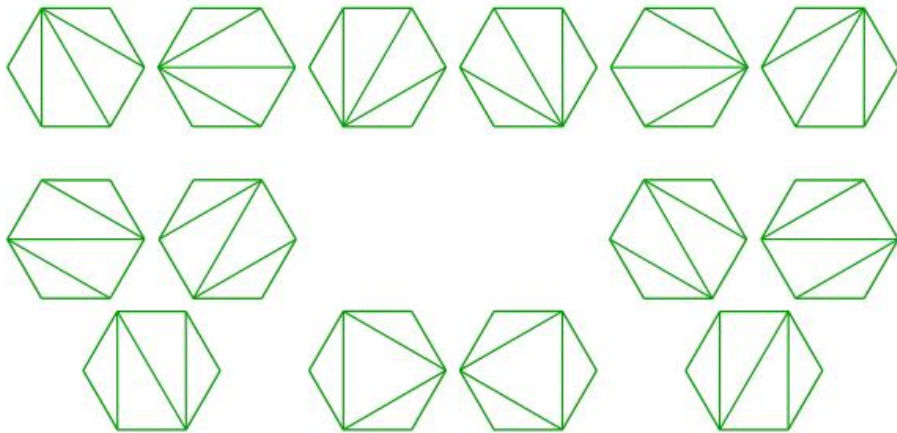
- Catalan numbers count the ways to arrange items according to specific rules, such as pairing or grouping

Parentheses $((())), ()(()), ()()(), (())(), (())()$.

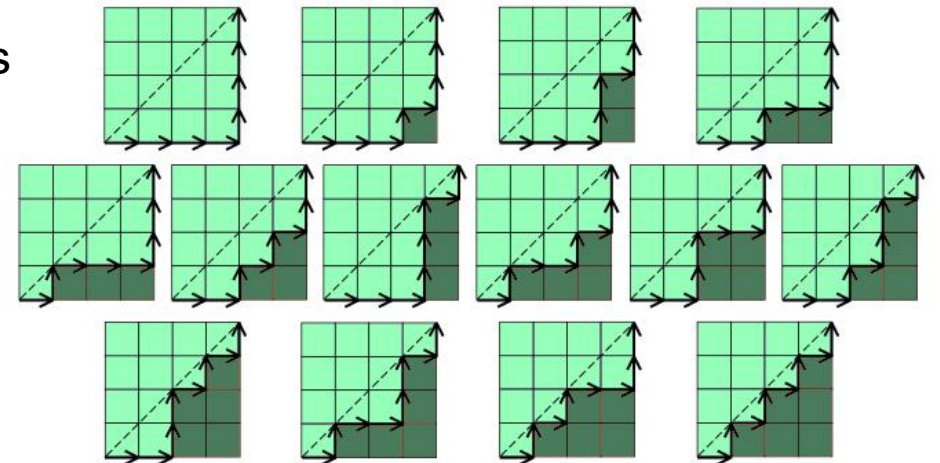


Binary search trees

Polygons



Paths



Optimal Substructure

- Imagine a split dividing the chain into two parts at index k :

$$(A_1 \times \cdots \times A_k) \times (A_{k+1} \times \cdots \times A_n)$$

- For the total cost to be minimal, two things must be true:
 1. The cost of calculating the left prefix $(A_1 \times \cdots \times A_k)$ must be the minimum possible cost
 2. The cost of calculating the right suffix $(A_{k+1} \times \cdots \times A_n)$ must be the minimum possible cost

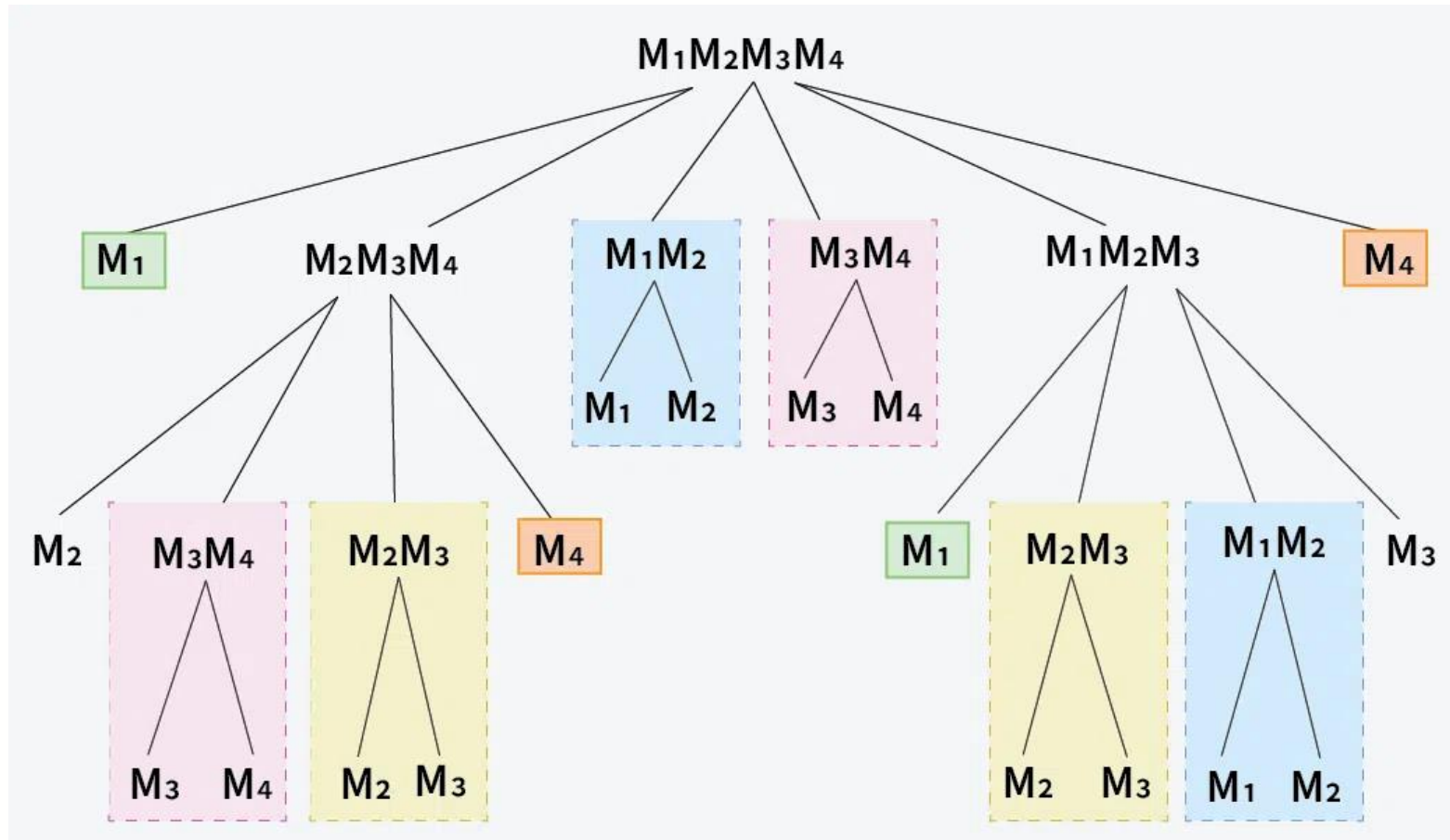


image:geeksforgeeks

- Overlapping Subproblems
- Optimal Substructure: optimal solution depends on the optimal solutions of smaller subproblems

Exercise

- For $A_1 A_2 \dots A_n, < p_0, p_1, \dots, p_n >$
- Imagine a split dividing the chain into two parts at index k :

$$(A_1 \times \dots \times A_k) \times (A_{k+1} \times \dots \times A_n)$$

- Where can we place the split k for a chain of length n (from A_1 to A_n)?
- For a split at k , what are the dimensions of the left prefix and the right suffix?
- What is the total cost formula for a specific choice of k ?

Exercise: Generalize

- Imagine a split dividing the chain into two parts at index k :

$$(A_i \times \cdots \times A_k) \times (A_{k+1} \times \cdots \times A_j)$$

- Where can we place the split k for a chain of length n (from A_i to A_j)?
- For a split at k , what are the dimensions of the left prefix and the right suffix?
- What is the total cost formula for a specific choice of k ?

Naive Recursive Algorithm

1. Evaluate all possible ways to parenthesize the product $A_1 A_2 \dots A_n$
 - Generate all possible ways to group the matrices in pairs (=every possible order in which the multiplications can be performed)
2. Calculate cost for each parenthesization
3. Select the minimum cost

Naive Recursive Algorithm

- Base case
 - Single matrix: no multiplication/addition required
 - $\text{min_cost}(A_i)=0$
- Formula:
 - $\text{min_cost}(A_i A_{i+1} \dots A_j) = \min_{i \leq k < j} (\text{min_cost}(A_i \dots A_k) + \text{min_cost}(A_k \dots A_j) + p_i p_k p_j)$ for $i < j$

Naive Recursive Algorithm

- Evaluate all possible ways to parenthesize the product $A_1 A_2 \dots A_n$
- If we have just 1 item, then there is only one way to parenthesize
- If we have n items, then there are $n-1$ places where you could break the list with the outermost pair of parentheses, namely just after the first item, just after the 2nd item, etc. and just after the $(n-1)$ th item.
- When we split just after the k th item, we create two sub-lists to be parenthesized, one with k items and the other with $n-k$ items. Then we consider all ways of parenthesizing these. If there are L ways to parenthesize the left sub-list, R ways to parenthesize the right sub-list, then the total possibilities is $L \cdot R$.

Naive Recursive Algorithm

```
def matrix_chain_recursive(p, i, j):  
    if i == j:  
        return 0 # No multiplication needed for a single matrix  
  
    min_cost = sys.maxsize # Initialize with a large value  
  
    for k in range(i, j):  
        cost = (matrix_chain_recursive(p, i, k) +  
                matrix_chain_recursive(p, k+1, j) + p[i-1] * p[k] * p[j])  
  
        min_cost = min(min_cost, cost) # Keep track of minimum cost  
  
    return min_cost
```

written by CoPilot

Naive Algorithm

- Recurrence Relation:

$$T(n) = \sum_{k=1}^{n-1} (T(k) + T(n - k)) + O(1)$$

- The number of subproblems at any level follows the Catalan number growth, leading to exponential complexity.
- Approximately, $T(n) \approx O(2^n)$ in the worst case, and more refined analysis often yields $O(n!)$

DP Solution

- State:

$dp[i, j]$ = Minimum cost to compute the product $A_i A_{i+1} \dots A_j$

- Base case:

- Single matrix multiplication: $dp[i, i] = 0$ for all i

DP Solution - Memoization

```
def matrix_chain_recursive(p, i, j, memo):  
    if i == j:  
        return 0 # No multiplication needed for a single matrix  
  
    if memo[i][j] != -1:  
        return memo[i][j]  
  
    min_cost = float('inf') # Initialize with a large value  
  
    for k in range(i, j):  
        cost = (matrix_chain_recursive(p, i, k) +  
                matrix_chain_recursive(p, k+1, j) + p[i-1] * p[k] * p[j])  
  
        min_cost = min(min_cost, cost) # Keep track of minimum cost  
        memo[i][j] = min_cost  
  
    return min_cost
```

DP Solution

- State:
 - $dp[i, j]$ = Minimum cost to compute the product $A_i A_{i+1} \dots A_j$
- Base case:
 - Single matrix multiplication: $dp[i, i] = 0$ for all i
- Tabulation: Bottom-up process
 - Base cases: Compute all chains of length 1 (e.g., $dp[1,1]$, ..., $dp[n,n]$)
 - Length 2: Compute costs for all possible two-matrix multiplications (e.g., $dp[1,2]$, $dp[2,3]$)
 - Length 3: Compute costs for all possible three-matrix multiplications (e.g., $dp[1,3]$, $dp[2,4]$)
 - Repeat until you reach length n
 - Final answer is stored in $dp[1,n]$

Example: Tabulation Method for MCM

- The initial set of dimensions are $\langle 5, 4, 6, 2, 7 \rangle$:
we are multiplying A_1 (5x4) times A_2 (4x6) times A_3 (6x2) times A_4 (2x7).
- Cost of multiplication $m[i,j]$: minimum number of scalar multiplications needed to multiply matrices from i to j
- $m[1,1]$, $m[2,2]$, $m[3,3]$, $m[4,4]$?

	1	2	3	4
1	0			
2		0		
3			0	
4				0

m

Example: DP for CMM

- The initial set of dimensions are $\langle 5, 4, 6, 2, 7 \rangle$: we are multiplying A_1 (5x4) times A_2 (4x6) times A_3 (6x2) times A_4 (2x7).
- $A_1 A_2$
 - $m[1,2] = 5 \times 4 \times 6 = 120$
- $A_2 A_3$
 - $m[2,3] = 4 \times 6 \times 2 = 48$
- $A_3 A_4$
 - $m[3,4] = 6 \times 2 \times 7 = 84$

m		1	2	3	4
1		0	120		
2			0	48	
3				0	84
4					0

Example: DP for CMM

A_1 (5x4), A_2 (4x6), A_3 (6x2), A_4 (2x7)

- $m[1,3] = \text{min_cost}(A_1 A_2 A_3)$
 - Cost of $((A_1 A_2) A_3) = m[1,2] + m[3,3] + p_0 * p_2 * p_3$
 $= 120 + 0 + 5 * 6 * 2 = 180$
 - Cost of $(A_1 (A_2 A_3)) = m[1,1] + m[2,3] + p_0 * p_1 * p_3$
 $= 0 + 48 + 5 * 4 * 2 = 88$
- $m[2,4] = \text{min_cost}(A_2 A_3 A_4)$
 - Cost of $((A_2 A_3) A_4) = m[2,3] + m[4,4] + p_1 * p_3 * p_4 = 104$
 - Cost of $(A_2 (A_3 A_4)) = m[2,2] + m[3,4] + p_1 * p_2 * p_4 = 252$

	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0

m

Example: DP for CMM

A_1 (5x4), A_2 (4x6), A_3 (6x2), A_4 (2x7)

- $m[1,4] = \text{min_cost}(A_1 A_2 A_3 A_4)$

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

m

$$\underbrace{A_1}_{m[1,1]} \mid \underbrace{A_2 A_3 A_4}_{m[2,4]}$$

$$m[1,1] + m[2,4] + p_0 * p_1 * p_4 = 244$$

$$\underbrace{A_1 A_2}_{m[1,2]} \mid \underbrace{A_3 A_4}_{m[3,4]}$$

$$m[1,2] + m[3,4] + p_0 * p_2 * p_4 = 414$$

$$\underbrace{A_1 A_2 A_3}_{m[1,3]} \mid \underbrace{A_4}_{m[4,4]}$$

$$m[1,3] + m[4,4] + p_0 * p_3 * p_4 = 158$$

DP for CMM

- $A_1 (p_0 \times p_1), A_2 (p_1 \times p_2), \dots, A_j (p_{j-1} \times p_j)$

- $m[1, j] = \min_cost(A_1 A_2 \dots A_j)$

$$\begin{array}{c} k=1 \\ A_1 | A_2 \dots A_j \\ \underbrace{\hspace{1cm}} \quad \underbrace{\hspace{1cm}} \\ m[1,1] \quad m[2,j] \end{array}$$

$$\begin{array}{c} k=2 \\ A_1 A_2 | A_3 \dots A_j \\ \underbrace{\hspace{1cm}} \quad \underbrace{\hspace{1cm}} \\ m[1,2] \quad m[3,j] \end{array}$$

$$\begin{array}{c} k=3 \\ A_1 A_2 A_3 | A_4 \dots A_j \\ \underbrace{\hspace{1.5cm}} \quad \underbrace{\hspace{1.5cm}} \\ m[1,3] \quad m[4,j] \end{array}$$

$$\begin{array}{c} k \\ A_1 \dots A_k | A_{k+1} \dots A_j \\ \underbrace{\hspace{1.5cm}} \quad \underbrace{\hspace{1.5cm}} \\ m[1,k] \quad m[k+1,j] \end{array}$$

$$m[1, j] = \min_{i \leq k < j} (m[1, k] + m[k + 1, j] + p_1 p_k p_j) \text{ for } 1 < j$$

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k + 1, j] + p_i p_k p_j) \text{ for } i < j$$

Matrix-Chain-Order(p)

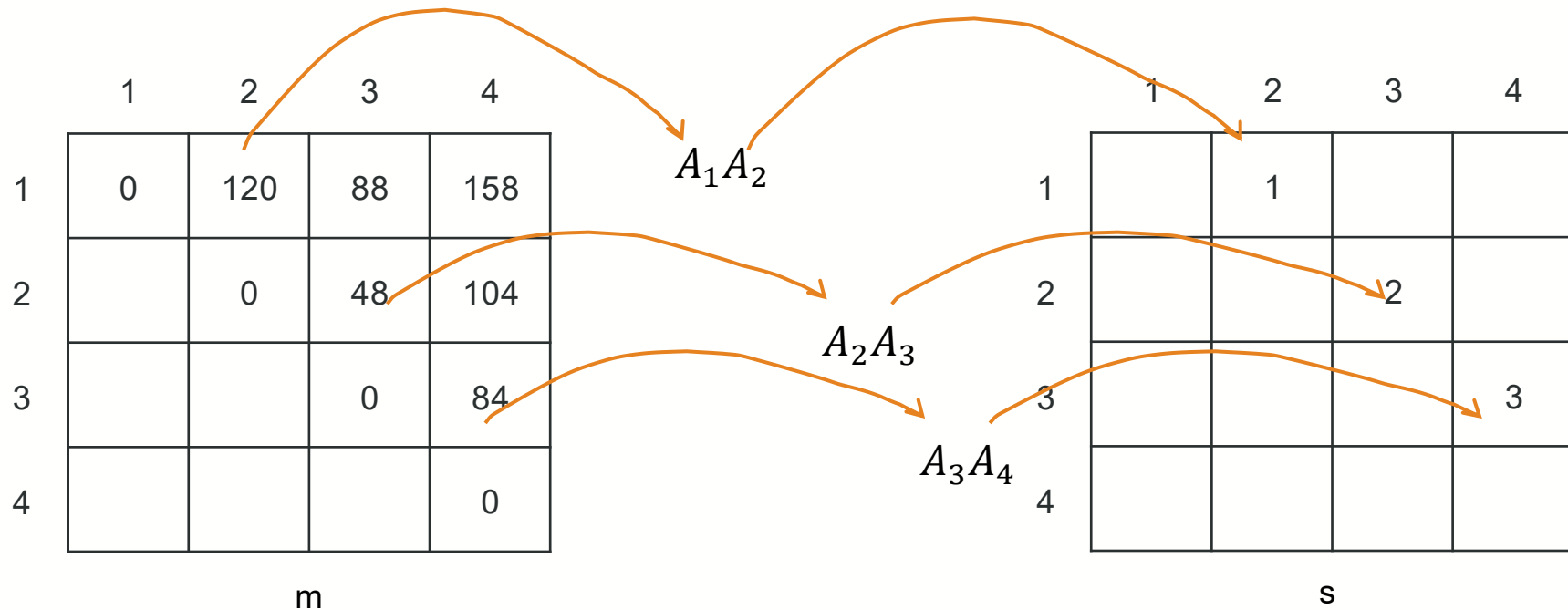
```
1.  $n \leftarrow \text{length}[p] - 1$ 
2. for  $i \leftarrow 1$  to  $n$                                 // initialization:  $O(n)$  time
3.   do  $m[i, i] \leftarrow 0$ 
4. for  $L \leftarrow 2$  to  $n$                              //  $L$  = length of sub-chain
5.   do for  $i \leftarrow 1$  to  $n - L + 1$ 
6.     do  $j \leftarrow i + L - 1$ 
7.      $m[i, j] \leftarrow \infty$ 
8.     for  $k \leftarrow i$  to  $j - 1$ 
9.       do  $q \leftarrow m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$ 
10.      if  $q < m[i, j]$ 
11.        then  $m[i, j] \leftarrow q$ 
12.         $s[i, j] \leftarrow k$ 
13. return  $m$  and  $s$ 
```


Analysis

- The array $s[i, j]$ is used to extract the actual sequence (see example).
- There are 3 nested loops and each can iterate at most n times, so the total running time is $\Theta(n^3)$.

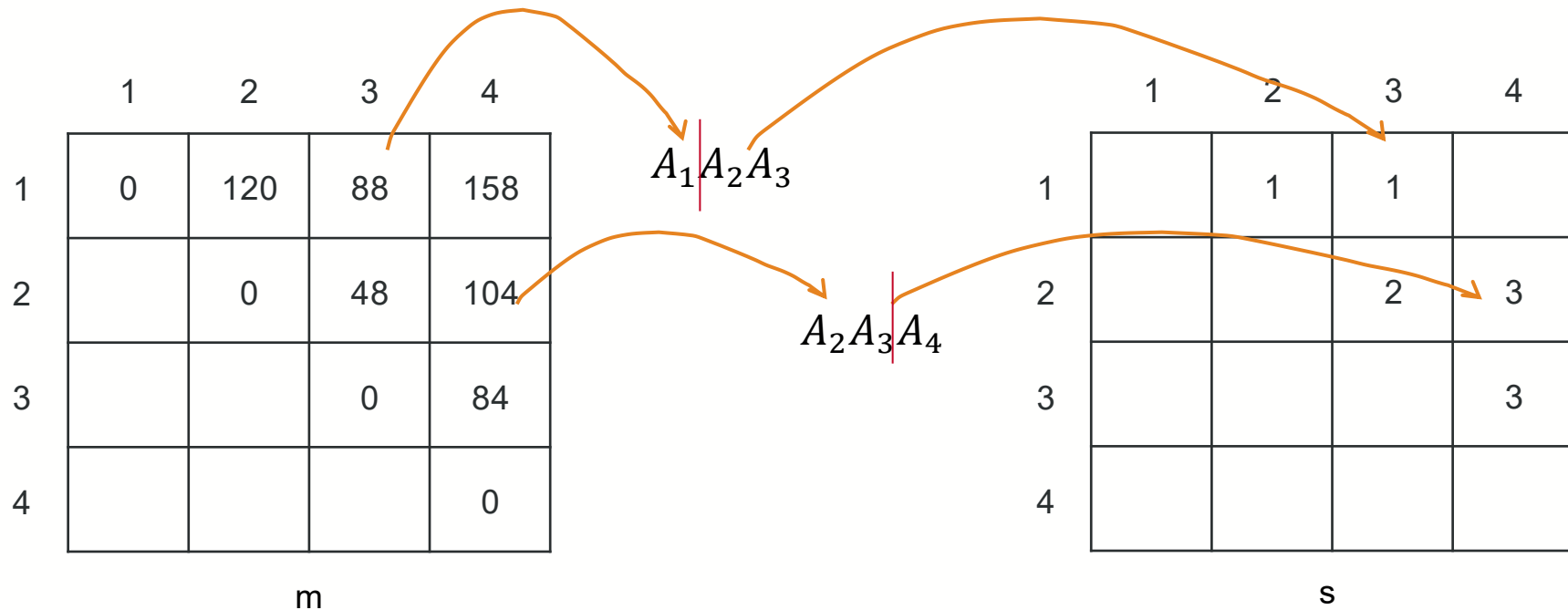
Example: DP for CMM – Optimal Sequence

- During the execution, save a split marker indicating where the best split is



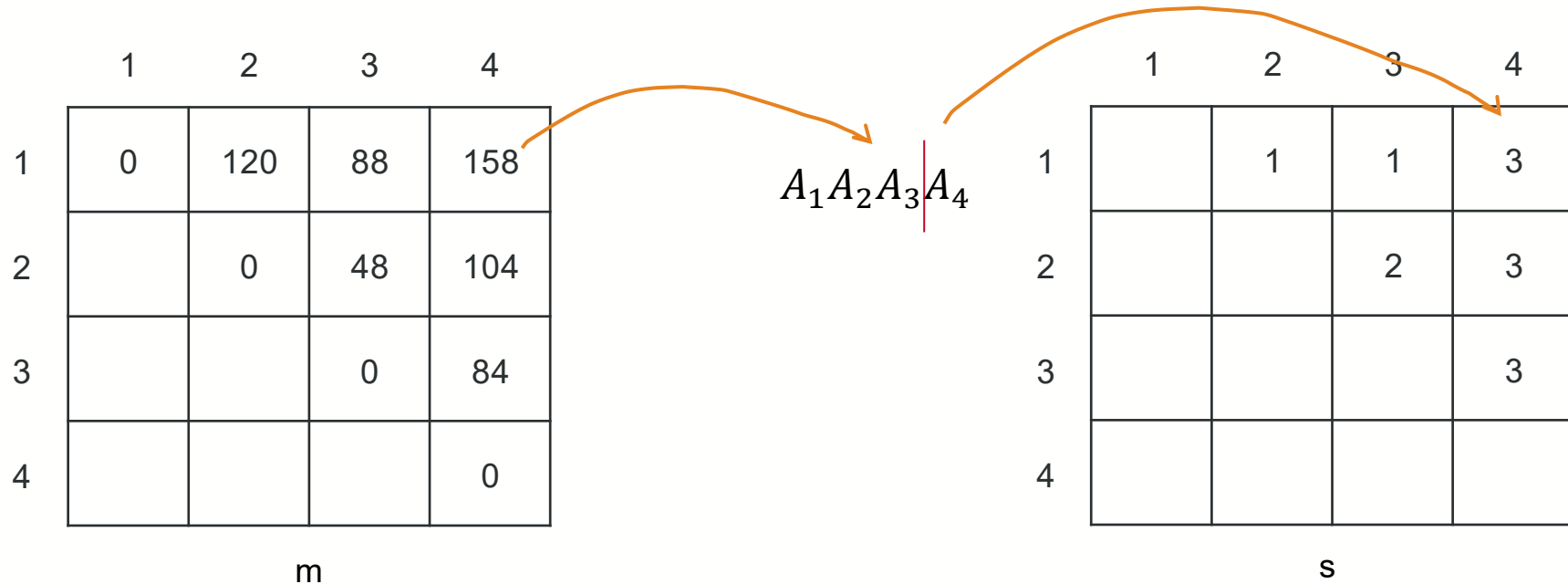
Example: DP for CMM – Optimal Sequence

- During the execution, save a split marker indicating where the best split is



Example: DP for CMM – Optimal Sequence

- During the execution, save a split marker indicating where the best split is



Exercise: Reading the DP Tables

- What is the minimum cost of to multiply the entire chain?
- What does the value in $m[i,j]$ represent?
- What does the value in $s[i,j]$ represent?
- How do you find the first split point for the full chain?
- Why is $m[i,i]=0$ for all i ?
- Why is the bottom-left half of the tables empty (or undefined)?

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

m

	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

s

Example: DP for CMM – Optimal Sequence

- The initial set of dimensions are $\langle 5, 4, 6, 2, 7 \rangle$: we are multiplying A_1 (5x4) times A_2 (4x6) times A_3 (6x2) times A_4 (2x7). Optimal sequence is $(A_1 (A_2 A_3)) A_4$.

$s[1,4]=3 \rightarrow A_{1\dots 3}A_4$

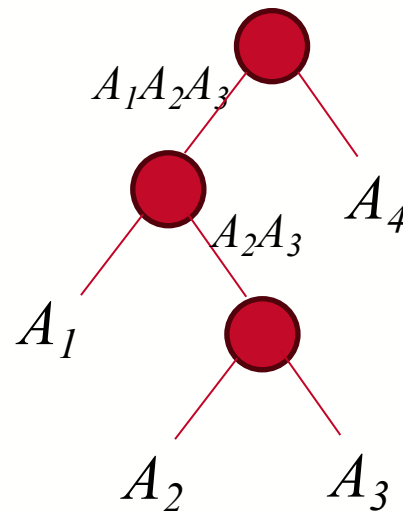
$A_1A_2A_3 \vdots A_4$

$s[1,3]=1 \rightarrow A_1A_{2\dots 3}$

$A_1 \vdots A_2A_3$

$s[2,3]=2 \rightarrow A_2A_3$

$A_2 \vdots A_3$



	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

s

Mult (A, i, j)

1. if ($j > i$)
2. **then** $k = s[i, j]$
3. $X = \text{Mult}(A, i, k)$ // $X = A[i] \dots A[k]$
4. $Y = \text{Mult}(A, k+1, j)$ // $Y = A[k+1] \dots A[j]$
5. return $X*Y$ // Multiply $X*Y$
6. **else** return $A[i]$ // Return i th matrix

Try Implementing Your Solution

- <https://www.geeksforgeeks.org/problems/matrix-chain-multiplication0303/1>

Longest Common Subsequence

What is subsequence

- A string generated from the original string by deleting 0 or more characters and without changing the relative order of the remaining characters.
- $X = \text{ABCDEFGF}$
 - Subsequence 1 of $X = \text{ABCFG}$
 - Subsequence 2 of $X = \text{ABCDEFGF}$
 - Subsequence 3 of $X = (\text{empty subsequence})$
 - Not a subsequence of $X = \text{ADC, GA, GFEDCBA...}$

Maximum subsequence problem (Longest Common Subsequence)

- Given sequence X and Y
- Find the longest possible sequence Z that is both subsequence of X and Y
- EX:
 - X = ABCDEFG
 - Y = CADZEG
 - Z = ADEG or CDEG

Why do we care?

- A measure of how similar two sequences are
- DNA and protein sequencing: Compare genetic sequences
- Text comparison: Plagiarism detection, diff tools
- File compression: e.g., LZW compression (use repeating patterns in data)
- Network security: Analyze network traffic for suspicious activity by identifying malicious subsequences of actions within a larger sequence of normal activity.
- Pattern recognition, NLP, video processing, etc.

Subsequence vs. Substring

- A substring must use consecutive characters from the original string
- $X = \text{ABCDEFGG}$
 - Substring 1 of $X = \text{ABCDE}$
 - Substring 2 of $X = \text{ABCDEFGG}$
 - Substring 3 of $X = (\text{empty subsequence})$
 - Not a Substring of $X = \text{ABCFG, ADC, GA, GFEDCBA...}$
- Substring $\subseteq$ subsequence
- Longest Common Substring problem: often solved with two pointers/sliding window (faster and simpler)

Longest Common Subsequence vs. Substring

- For each scenario, choose the best answer and justify your choice.
(a) LCS (b) LCSubstr (c) Both (d) Neither
- 1. Two documents contain the exact same 15-word sentence word-for-word. Which method(s) will reliably detect this?
- 2. A student reorders clauses within a paragraph (e.g., swapping the first and last half of a sentence). Which method breaks down and fails to show a significant match?
- 3. A student takes a paragraph and deletes every 5th word or changes a few verb tenses to hide their tracks. Which method is better at catching this “gappy” similarity?
- 4. Two documents share many common characters and small words (e.g., “the”, “a”, “is”) but have no meaningful relation. Which method might misleadingly report a high similarity score?
- 5. Two documents express the same core idea but use completely different vocabulary and synonyms (e.g., “The feline rested” vs. “The cat slept”). Which method detects this similarity?

Optimal Substructure

- Imagine a split dividing the chain into two parts at index k :

$$(A_1 \times \cdots \times A_k) \times (A_{k+1} \times \cdots \times A_n)$$

- For the total cost to be minimal, two things must be true:
 1. The cost of calculating the left prefix $(A_1 \times \cdots \times A_k)$ must be the minimum possible cost
 2. The cost of calculating the right suffix $(A_{k+1} \times \cdots \times A_n)$ must be the minimum possible cost

Brute force approach

- Enumerate all subsequence of X
 - Check if it's a subsequence of Y
 - Keep track of the longest subsequence

EX:

X = ABCDEFG

Y = CADZEG

C, A, D, Z, E, G, CA, CD, CZ, CE, CG, AD, AZ, AE, AG, DZ, DE, DG, ZE, ZG, EG, CAD, CAZ, CAE, CAG, CDZ, CDE, CDG, CZE, CZG, CEG, ADZ, ADE, ADG, AZE, AZG, AEG, DZE, DZG, DEG, ZEG, CADZ, CADE, CADG, CAZE, CAZG, CAEG, CDZE, CDZG, CDEG, CZEG, ADZE, ADZG, ADEG, AZEG, DZEG, CADZE, CADEG, CADZG, CAZEG, CDZEG, ADZEG, CADZEG

Brute force approach

- Assume X has m element $\{x_1, x_2, x_3, \dots, x_m\}$
 - Each subsequence must choose whether or not to include element i
 - 2 for each element
 - 2^m possible subsequence for X
- ~Exponential time....slow!

Optimal Substructure: Subproblems

- The sub-problems correspond to pairs of prefixes
- Define prefix function:
 - Given $X = \{x_1, x_2, x_3, \dots, x_m\}$
 - Define the i th prefix of X as **X_i** = $\{x_1, x_2, x_3, \dots, x_i\}$

Optimal Substructure: Subproblems

- Consider a sequence $X = \text{"ABCDE"}$
- The 1st prefix x_1 is "A".
- The 2nd prefix x_2 is "AB".
- The 3rd prefix x_3 is "ABC".
- And so on, up to the m-th prefix x_m .

Optimal Substructure: Subproblems

- X_i = prefix function of X
- x_i = i th element of X
- Let $X = \{x_1, x_2, x_3, \dots, x_m\}$ $Y = \{y_1, y_2, y_3, \dots, y_n\}$
- State: $dp[i][j]$
 - The length of the LCS between the prefix X_i (i.e., $X[1 \dots i]$) and the prefix Y_j (i.e., $Y[1 \dots j]$)
- At any given state (i, j) , you are looking at the current characters x_i and y_j . You have two primary scenarios that dictate your choices:
 - The characters match ($x_i == y_j$)
 - Pick this character and add 1 to your count
 - New subproblem: $dp[i-1][j-1]$
 - The characters do not match ($x_i \neq y_j$)
 - Take the maximum of two subproblems: $dp[i][j-1]$ or $dp[i-1][j]$

LCS Base Case

- The base case for the LCS occurs when either of the two strings is empty
- $dp[i][j]=0$, if $i=0$ or $j=0$

Case 1 Example

- $X = \text{ABCDEFGG}$
- $Y = \text{BDCDEFG}$
- Compare x_7 and $y_7 \rightarrow G == F$
- When the characters match, you include this character in your optimal subsequence

$$\text{DP}[7][7] = 1 + \text{DP}[6][6]$$

- where $\text{dp}[6][6] = \text{LCS of } x_6 (\text{ABCDEF}) \text{ and } y_6 (\text{BDCDEF})$

Case 2 and Case 3 Example

- $X=ABCDEFGG, Y=BDCDEF$
- Compare x_7 and $y_6 \rightarrow G \neq F$
- $DP[7][6]$ = must be the max of:
 - Case 2:
 - Drop x_7 : $DP[6][6] = \text{LCS}(ABCDEF, BDCDEF)$
 - Case 3:
 - Drop y_6 : $DP[7][5] = \text{LCS}(ABCDEFGG, BDCDE)$

Optimal Substructure: Subproblems (formally)

- X_i = prefix function of X
- x_i = ith element of X
- Let $X = \{x_1, x_2, x_3, \dots, x_m\}$ $Y = \{y_1, y_2, y_3, \dots, y_n\}$
- Let $Z = \{z_1, z_2, z_3, \dots, z_m\}$ be the LCS of X and Y
- 3 cases to consider
 - If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is LCS of X_{m-1} and Y_{n-1}
 - If $x_m \neq y_n$, then $z_k \neq x_m$ implies Z_k is LCS of X_{m-1} and Y
 - If $x_m \neq y_n$, then $z_k \neq y_n$ and Z_k is LCS of X and Y_{n-1}

Putting it together in a recursive solution

- Given $X = \{x_1, x_2, x_3, \dots, x_m\}$ $Y = \{y_1, y_2, y_3, \dots, y_n\}$
- If $x_m = y_n$
 - Find LCS of (X_{m-1}, Y_{n-1})
 - Append x_m to it to get LCS of (X, Y)
- Else if $x_m \neq y_n$
 - Find LCS of (X_{m-1}, Y) and LCS of (X, Y_{n-1})
 - The longest one will be the LCS of (X, Y)
- These cases exhaust all possibilities, thus optimal sub-problem solution must appear in the optimal solution to the problem

Define recursion

- $C[i,j]$ = length of the LCS of **X_i** and **Y_j**

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 , \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j , \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j . \end{cases}$$

Brute Force Recursive Solution

$i = \text{len}(X) - 1, j = \text{len}(Y) - 1$

```
def LCS(i, j):  
    if i < 0 or j < 0:  
        return 0           //no common subsequence  
    if X[i] == Y[j]:  
        return 1 + LCS(i-1, j-1)  
    else:  
        return max(LCS(i-1, j), LCS(i, j-1))
```

Recursive Solution with Memoization

```
def LCS(i, j):
```

```
    if i<0 or j<0: return 0
```

```
    //check memo
```

```
    if (i, j) in memo: return memo[(i,j)]
```

```
    if X[i]==Y[j]:
```

```
        memo[(i,j)] = 1+LCS(i-1, j-1)
```

```
    else:
```

```
        memo[(i,j)] = max(LCS(i-1, j), LCS(i, j-1))
```

```
    return memo[(i,j)]
```

- Each unique pair (i, j) is computed once
- There are $m*n$ such pairs

→ $O(m*n)$

Algorithm - Tabulation

- $C[i,j] = C[i,j]$ = length of the LCS of X_i and Y_j
- $B[i,j]$ = points to the optimal subproblem
- Time complexity?
 - $O(mn)$

LCS-LENGTH(X, Y)

```
1   $m = X.length$ 
2   $n = Y.length$ 
3  let  $b[1..m, 1..n]$  and  $c[0..m, 0..n]$  be new tables
4  for  $i = 1$  to  $m$ 
5       $c[i, 0] = 0$ 
6  for  $j = 0$  to  $n$ 
7       $c[0, j] = 0$ 
8  for  $i = 1$  to  $m$ 
9      for  $j = 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] = c[i - 1, j - 1] + 1$ 
12              $b[i, j] = "\nwarrow"$ 
13         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] = c[i - 1, j]$ 
15              $b[i, j] = "\uparrow"$ 
16         else  $c[i, j] = c[i, j - 1]$ 
17              $b[i, j] = "\leftarrow"$ 
18  return  $c$  and  $b$ 
```

Example

- <https://www.cs.usfca.edu/~galles/visualization/DPLCS.html>

Exercise: Reading the DP Table

- $X = \{A, B, C, B, D, A, B\}$
- $Y = \{B, D, C, A, B, A\}$
- What is the length of the LCS?
- What does the value in $C[i, j]$ represent?
- What does the arrow in $B[i, j]$ indicate?
- How do you reconstruct the LCS?
- Why are the first row and first column of C initialized to 0?

		j						
		0	1	2	3	4	5	6
i	y_j		B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑	↑	↑	↖	←	↖
2	B	0	↖	←	←	↑	↖	←
3	C	0	↑	↑	↖	←	↑	↑
4	B	0	↖	↑	↑	↑	↖	←
5	D	0	↑	↖	↑	↑	↑	↑
6	A	0	↑	↑	↑	↖	↑	↖
7	B	0	↖	↑	↑	↑	↖	↑

Table C and B combined

Summary for DP

- DP really just means keep the intermediate solution to avoid recomputing it again
- A trade off of storage for speed
- General strategy
 - Find substructure property
 - Build a table, matrix, cube, etc that represent it
 - Start filling in the trivial cases
 - Then fill in the next level that calls upon those trivial case
 - Then fill in the next level using whats already filled in
 - Rinse and repeat (Iterate until the entire problem is solved)
 - Profit?
 - Efficiency (by avoiding redundant calculations)

Variants of LCS

- Longest Increasing Subsequence (LIS)
 - Longest subsequence where each element is greater than the preceding one.
 - Example:
 - Given the sequence: 10, 22, 9, 33, 21, 50, 41, 60, 80
 - The LIS is: 10, 22, 33, 50, 60, 80 (length 6)
- Applications in Machine Learning
 - Feature engineering for sequential data
 - Time series analysis (e.g., stock price predictions, anomaly detection)
 - DNA, RNA, protein sequence mining

Variants of LCS

- Edit Distance (Levenshtein distance)
 - Minimum number of operations (insertions, deletions, or substitutions) required to transform one string into the other
 - Example:
 - String 1: "kitten"
 - String 2: "sitting"
 - Edit Distance: 3 (Operations: kitten → sitten → sittin → sitting)
- Applications in Machine Learning
 - Natural Language Processing (NLP): spell checking, text similarity (plagiarism detection, document clustering)
 - Voice recognition (comparing recognized text with possible correct phrases)

Edit Distance

- Given two strings word1 and word2, return *the minimum number of operations required to convert word1 to word2*.
- You have the following three operations permitted on a word:
 - Insert a character
 - Delete a character
 - Replace a character

Constraints:

$0 \leq \text{word1.length}, \text{word2.length} \leq 500$

word1 and word2 consist of lowercase English letters.

Example 1:

Input: word1 = "horse", word2 = "ros"

Output: 3

Explanation:

horse -> rorse (replace 'h' with 'r')

rorse -> rose (remove 'r')

rose -> ros (remove 'e')

Example 2:

Input: word1 = "intention", word2 = "execution"

Output: 5

Explanation:

intention -> inention (remove 't')

inention -> enention (replace 'i' with 'e')

enention -> exention (replace 'n' with 'x')

exention -> exection (replace 'n' with 'c')

exection -> execution (insert 'u')